Certified Worst-Case Honeypot Scheduling Under Decoy Recognition: Closing Four Gaps in Attack-Graph Placement

Anonymous Authors (for double-blind review)

***Abstract*—Recent honeypot-placement work places decoys on attack graphs with mixed-integer programs, Bayesian Stackelberg games and deep reinforcement learning, and re-optimizes when alerts arrive. Attack-path coverage, early interception, exact placement and multiple attacker types are therefore established. Four gaps remain: a placed decoy is assumed to stay unrecognized; a decoy is a location without an identity; placement is re-solved per round without rules that span time slots; and the objective is an expected or learned payoff without a checkable optimality proof. We present ZSTP, a weighted partial MaxSAT formulation that closes all four. It plans trap type, zone, time slot and persona jointly; models per-decoy type and fingerprint recognition for each of four attacker profiles; enforces identity, churn and critical-hop frequency and gap rules as hard constraints; and computes the max–min schedule $r^\star$ with a refutation proof verified by an independent checker. On a 640-decision instance, $r^\star = 7{,}052.62$. Each contribution is tested against each of ten baselines with gap-specific statistics in the model and in an emulation over real TCP sockets with common random numbers: every baseline has its decoys recognized significantly more often (G1, G2), intercepts significantly fewer attacks at the critical hop (G3), and stays below the certified worst case (G4). Per-gap ablations show that leaving recognition, identity or multi-slot planning out of the model cuts the worst case to 42.0–55.5% of $r^\star$, while an expected-payoff objective matches $r^\star$ only when its prior favours the binding profile.**

***Index Terms*—Honeypot scheduling, cyber deception, MaxSAT, adversarial robustness, decoy recognition, optimality certificate, attack graphs.**

I. Introduction

A decoy is useful only until the attacker recognizes it. An attacker who sees the same trap type in the same zone in consecutive observation periods, or the same host identity in two places, can discount that decoy, and a schedule evaluated as if decoys were never recognized overstates its value. In our evaluation, a fast attacker recognizes 50–75% of the honeypots placed by ten baseline heuristics.

Honeypot placement has recently moved onto attack graphs, and the peer-reviewed placement literature of 2026 already solves much of the spatial problem. TrapManager ranks critical attack paths with a graph attention network and places decoys with a budget-constrained mixed-integer program that is re-solved when a trap fires [1]; a weighted core attack graph prunes attack paths for honeypot allocation against lateral movement [2]; Bayesian Stackelberg formulations allocate honeypots against several attacker types and update beliefs from intrusion-detection observations [3], [4]; and a Stackelberg game combined with deep reinforcement learning adjusts software-defined honeypot deployments under a budget [5]. We do not claim attack-path coverage, early interception, exact placement, multiple attacker types or alert-driven re-optimization as contributions.

To our knowledge, these works leave four gaps, which this paper names and closes:

- **G1 Recognition.** A placed decoy is treated as unrecognized until it is engaged [1], [3].
- **G2 Identity.** A decoy is a location without a separate identity, and nothing limits reuse of an identity across zones or slots [1], [3].
- **G3 Temporal planning.** Placement is re-solved per round or per alert [1], [3], with critical-path coverage required only within a single placement [1].
- **G4 Assurance.** The objective is an expected payoff under a prior over attacker types [3], [4], a learned deployment [5] or a single-attacker program [1]; no machine-checkable proof of optimality is reported.

The gaps interact. A schedule that is best against a slow attacker can be recognized quickly by a fast one; a decoy that keeps its location but changes its identity can remain useful where a repeated one cannot; and rules that keep a critical hop watched across slots force exactly the repetition that a fast attacker recognizes. On our instance, the schedule that is best for the medium attacker loses 33.1% of the attainable worst case, and zero recognition under all profiles is provably infeasible. Closing the gaps one at a time is therefore not enough; they must be modeled jointly.

Contributions. Each of the first four contributions closes one gap; the fifth tests each of them against every baseline.

1) *Per-decoy recognition inside an exact formulation (G1).* One Boolean per (trap type, zone, slot, persona), with type and fingerprint burn derived for each attacker profile and gating every credit term; the problem is NP-hard (Theorem 1).
2) *Decoy identity as a decision axis (G2).* A persona decomposition $P \subseteq \hat{R} \times F$ separates role plausibility from fingerprint, with hard identity rules; two fingerprints per role raise $r^\star$ by 28.8%.
3) *A multi-slot schedule with inter-slot hard rules (G3).* Churn and a split minimum-frequency/maximum-gap rule on every critical hop; zero recognition under all profiles is proven infeasible.
4) *A certified worst case (G4).* An exact SAT-based max–min search returns $r^\star$ with a refutation proof verified

by an independent checker; a threat-adaptive loop re-certifies after every intelligence update.

5) *A per-contribution evaluation.* For each gap, a gap-specific statistic measured in the model and in socket-level emulation with paired statistics against each of ten baselines, both as produced and after repair; per-gap ablations; and a validation suite, all regenerated by one command.

Section II positions ZSTP against the 2026 placement literature. Sections III and IV present the model and the certified procedures, and Section V works through a small example. Section VI evaluates each contribution against each baseline, Section VII discusses deployment and limits, and Section VIII concludes.

II. Related Work

Background. Honeypots originated as standalone decoy systems and virtual honeypot frameworks [7], [8], later extended to adaptive honeypots that change their interaction with the attacker [9]. Game-theoretic work models honeypot allocation as a strategic game [10], surveys defensive-deception games [11] and studies feedback-driven moving-target defense [12]; reinforcement learning has been used to learn engagement policies [13] and intrusion-prevention strategies [14]; and attack graphs have been used for optimal sensor placement [15]. The detailed comparison below is made against the more recent and closer 2026 placement work.

Attack-graph honeypot placement (2026). Liu *et al.* [1] (TrapManager) build a vulnerability-based attack graph, rank critical attack paths with a graph attention network, and place traps with a binary mixed-integer program that maximizes path-weighted interception under a budget and requires at least one trap on each of the top-K critical paths; when a trap fires, the program is re-solved. Their attacker model assumes that traps are not recognized before interaction, and adversaries that adapt to repeated alerts are left out of scope. Lacmou Zeutouo *et al.* [2] reduce the attack graph to a weighted core that ranks and preserves attack paths by exploit difficulty and asset criticality, keeping allocation tractable on networks of more than 300 nodes. Park and Dagher [3] allocate honeypots against several concurrent attackers, each with its own exploits, costs and targets, in a multi-follower Bayesian Stackelberg game; beliefs about attacker types are updated from intrusion-detection and honeypot observations, and a DOBSS-style mixed-integer program is re-solved each round. Voznyi *et al.* [4] formulate honeypot placement as a Bayesian Stackelberg game that couples detection with attacker-type inference.

Learning-based deployment (2026). Zhang *et al.* [5] combine a Stackelberg game with deep Q-learning for software-defined honeypots: binary deployment vectors are chosen under a budget on the number of honeypots, and the defense benefit of the Stackelberg optimum serves as a threshold for adjusting the deployment. The deployment is learned rather than proven optimal.

Positioning. Table I positions ZSTP against these papers, Table II maps each gap to the ZSTP element that closes it, and Table III compares ZSTP with each paper individually. ZSTP does not compete on attack-path modeling: it takes attack paths as input. Critical paths ranked by [1] or reduced by [2] could populate them, and the per-round re-solve of [3] could be replaced by the certified re-solve of Section IV-B.

TABLE I
Positioning against the peer-reviewed 2026 honeypot-placement papers (✓ addressed; ○ partly; × not addressed; · not discussed), characterized from each paper's published description.

Paper	*Attack paths*	*Exact solver*	*Rules beyond budget*	*Burn per decoy (G1)*	*Several types*	*Worst case + proof (G4)*	*Type×zone×time×persona*
Park & Dagher [3]	✓	✓	×	×	✓	×	×
Voznyi *et al.* [4]	·	·	·	·	✓	×	·
Liu *et al.* [1]	✓	✓	○	×	×	×	×
Lacmou Z. *et al.* [2]	✓	·	·	·	·	·	·
Zhang *et al.* [5]	✓	×	·	·	·	×	·
ZSTP	input	✓	✓	✓	✓	✓	✓

○: only a path-coverage rule beyond the budget [1]. "input": ZSTP does not claim attack-path modeling as new.

III. Problem Formulation

A. Decisions and Instance

The decision variable $x_{i,z,t,p} \in \{0,1\}$ deploys trap type $i \in K$ in zone $z \in Z$ at slot $t \in T = \{1,\dots,H\}$ as persona $p \in P$. The instance I fixes budgets B_t and B_z, a plausibility table G_K over (type, zone, role), conflicting type pairs C, air-gapped zones Z_{air} with the trap types forbidden there, a churn limit $c_{\max}$, a gap window W, attack paths Π, ATT&CK techniques [16] with tactic families and weights w_j, and four attacker profiles $\Theta = \{\theta_{\text{low}}, \theta_{\text{med}}, \theta_{\text{high}}, \theta_{\text{burst}}\}$ with attack strengths $\rho_\theta = 0.10, 0.30, 0.60, 0.85$ and path emphasis $\omega_{\theta,\pi}$. An attack path π is a chain of hops, each taken in one zone with one technique and an interception value; π has a frequency ρ_π and a critical hop, the step into the target zone. A decoy covers a hop when it is live in that zone and slot and its type detects the hop's technique. Fig. 1 illustrates paths, hops and the force multiplier, the number of attack-path hops a single decoy serves.

On the evaluated instance ($|K| = 8$, $|Z| = 5$, $H = 4$, $|P| = 4$) there are 640 decisions; the encoding has 8,340 variables and 33,011 hard clauses. Every derived variable is defined by a full equivalence, so it is fixed by x; we write $\text{pres}_{i,z,t} = \bigvee_p x_{i,z,t,p}$ and $\text{used}_{f,t} = \bigvee_{i,z,p:f(p)=f} x_{i,z,t,p}$.

B. Persona Decomposition (G2)

A persona $p = (r, f)$ pairs a role template $r \in \hat{R}$ (what the decoy pretends to be) with a fingerprint bundle $f \in F$ (hostname, account, certificate, banners, OS signature: what the attacker observes). Plausibility (C5b) reads only $r = \pi_1(p)$; identity rules and fingerprint burn read only $f = \pi_2(p)$.

TABLE II
THE FOUR GAPS, WHERE THEY APPEAR IN THE 2026 PLACEMENT LITERATURE, THE ZSTP ELEMENT THAT CLOSES EACH, AND WHERE EACH IS TESTED AGAINST THE BASELINES.

Gap	Where it appears	ZSTP element that closes it	Tested in
G1 Recognition	Traps assumed unrecognized [1]; entering a honeypot counts as detection, no recognition model [3]	Type burn (C9) and fingerprint burn (C13) per profile gate every credit term (Sec. III-D)	Sec. VI-C, Table VI; ablation A1
G2 Identity	A decoy is a location without an identity [1], [3]	Persona $p = (r, f)$; hard rules C12, C14; fingerprint rotation (Sec. III-B)	Sec. VI-D, Table VII; ablation A2
G3 Temporal planning	Re-solved per round or alert [1], [3]; path coverage within one placement [1]	Multi-slot horizon with churn C8 and critical-hop rules C10a/C10b (Sec. III-C)	Sec. VI-E, Table VIII; ablation A3
G4 Assurance	Expected payoff [3], [4]; learned deployment [5]; single-attacker program [1]	Max–min $r^\star$ with an independently verified refutation proof (Sec. IV-A)	Sec. VI-F, Table XI; ablation A4

TABLE III
ZSTP COMPARED WITH EACH PEER-REVIEWED 2026 HONEYPOT-PLACEMENT PAPER: WHAT THE PAPER ALREADY DOES (NOT CLAIMED BY ZSTP) AND WHAT ZSTP ADDS. RULE IDENTIFIERS REFER TO TABLE IV.

Paper	Already does (not claimed by ZSTP)	What ZSTP adds
Park and Dagher [3]	Attack-graph placement against several attacker types; Bayesian Stackelberg game solved as a DOBSS-style MILP each round, with beliefs updated from IDS observations	Per-decoy recognition (C9, C13); identity rules (C12, C14); inter-slot rules (C8, C10a, C10b) over a planning horizon; worst case over profiles with an independently checked proof instead of expected payoff under a prior
Voznyi *et al.* [4]	Bayesian Stackelberg honeypot placement coupled with attacker-type inference	Worst case over profiles instead of expected payoff; per-decoy burn; decoy identity as a decision axis
Liu *et al.* [1]	Critical attack paths ranked by a graph attention network; budgeted MIP placement with mandatory top-K path coverage; re-solve when a trap fires; early interception	Recognition per decoy and profile; identity rules; minimum-frequency and maximum-gap coverage over several slots instead of single-placement coverage; several attacker profiles; optimality proof
Lacmou Zeutouo *et al.* [2]	Weighted core attack graph that ranks and preserves attack paths for allocation against lateral movement	Complementary: its reduced paths could populate Π; ZSTP adds the time, identity and burn structure and the certified schedule
Zhang *et al.* [5]	Stackelberg game plus deep Q-learning to adjust software-defined honeypots under a budget	Exact, certified optimum rather than a learned deployment; per-decoy burn; identity rules; inter-slot rules over a planning horizon

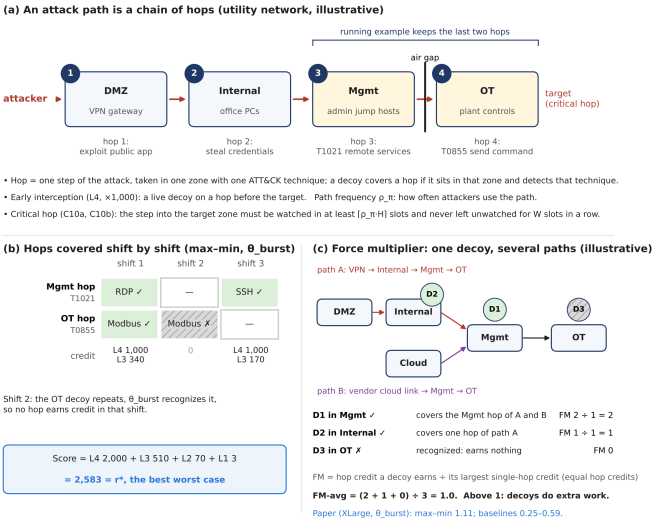


Fig. 1. Attack paths, hops and the force multiplier. (a) A four-hop path through an illustrative utility network; the step into OT is the critical hop. (b) Credits of a schedule hop by hop and slot by slot under θ_{burst}; a repeated OT decoy is recognized and earns nothing. (c) A decoy at a hop shared by two paths serves both.

Retiring a fingerprint therefore does not retire the role, so a role can keep covering a zone under a rotated identity. Because no fingerprint may appear twice in one slot, at most $|F|$ honeypots run per slot. In the main experiments each of four roles is bound to its own fingerprint ($|P| = |F| = 4$, which is also B_t); Section VI-D gives each role two.

C. Hard Rules (G2, G3)

The hard layer is, for all t, z, f and paths π,

$$\sum_{i,z,p} x_{i,z,t,p} \le B_t, \qquad \sum_{i,p} x_{i,z,t,p} \le B_z, \tag{C2, C3}$$

$$\text{pres}_{a,z,t} + \text{pres}_{b,z,t} \le 1 \quad \forall (a,b) \in C, \tag{C4}$$

$$x_{i,z,t,p} = 0 \text{ if } (i, z, \pi_1(p)) \notin G_K \text{ or } (i,z) \text{ barred}, \tag{C5, C5b}$$

$$\sum_{t \ge 2} |x_{i,z,t,p} - x_{i,z,t-1,p}| \le c_{\max}, \tag{C8}$$

$$\sum_t y_{\pi,t} \ge \lceil \rho_\pi H \rceil, \tag{C10a}$$

$$\bigvee_{s=t}^{t+W-1} y_{\pi,s}, \tag{C10b}$$

$$\sum_{i,p:\, f(p)=f} x_{i,z,t,p} \le 1, \tag{C12}$$

$$\sum_{i,z,p:\, f(p)=f} x_{i,z,t,p} \le 1, \tag{C14}$$

where $y_{\pi,t}$ indicates that a deployed type in the critical-hop zone of π detects its technique at slot t. Cardinality rules are encoded with sequential counters; a hard rule is never traded for score. Table IV gives their meaning. Neither C10a nor C10b implies the other: coverage in slot 1 only can satisfy C10a and break C10b, and coverage in slot 2 only can satisfy C10b and break C10a.

TABLE IV
HARD RULES AND HOW MANY OF THE TEN BASELINES BREAK EACH AS PRODUCED (INDEPENDENT RULE CHECKER).

Rule	Meaning	Broken by
C2, C3	At most $B_t = 4$ honeypots per slot, $B_z = 2$ per zone and slot	0, 2
C4	No conflicting cover stories in one zone and slot	1
C5, C5b	Air gap; (type, zone, role) plausible	0, 0
C8	Each (type, zone, persona) switches on/off at most twice	2
C10a	Critical hop covered in $\geq \lceil \rho_\pi H \rceil$ slots	6
C10b	Critical hop never uncovered for $W = 2$ consecutive slots	7
C12	No fingerprint twice in one zone and slot	4
C14	No fingerprint in two zones in one slot	6

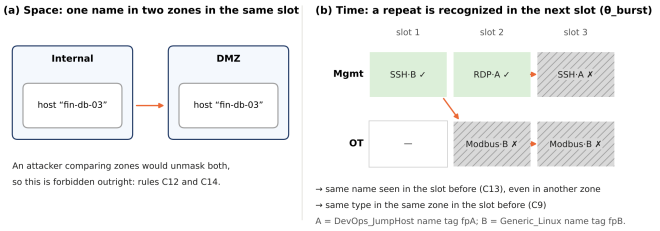


Fig. 2. How a decoy is recognized. (a) Space (G2): one identity in two zones in the same slot would unmask both decoys, so C12 and C14 forbid it. (b) Time (G1): under θ_{burst} a deployment is recognized when the same type stood in the same zone in the previous slot (C9) or its fingerprint was shown in the previous slot in any zone (C13); green cells earn credit, hatched cells do not.

D. Recognition (G1)

For profile θ, type burn and fingerprint burn are

$$u^{\theta}_{i,z,t} = \bigwedge_{s=t-\tau_d(\theta,z)}^{t-1} \text{pres}_{i,z,s}, \qquad v^{\theta}_{f,t} = \bigwedge_{s=t-\tau_{dp}(\theta)}^{t-1} \text{used}_{f,s}, \tag{1}$$

(false when the window starts before slot 1), and a deployment earns credit only while live:

$$d^{\theta}_{i,z,t,p} = x_{i,z,t,p} \wedge \neg u^{\theta}_{i,z,t} \wedge \neg v^{\theta}_{f(p),t}. \tag{2}$$

Recognition is decided only from earlier slots (observe-then-burn), so every decoy earns credit the first time it is seen. Fig. 2 illustrates both forms. The thresholds follow the threat update $\tau = \max(1, \lfloor \tau_0(1 - \rho_{\text{eff}}/\rho_{\max}) \rfloor)$ with $\tau_{d0} = 3$, $\tau_{dp0} = 4$ and $\rho_{\max} = 1$, where ρ_{eff} is the larger of the profile strength and the frequency of paths through the zone; this gives $\tau_d = (2, 2, 1, 1)$ and $\tau_{dp} = (3, 2, 1, 1)$ for the four profiles, so θ_{burst} recognizes any repeat in the next slot.

E. Objective and Robust Target (G4)

The score $Q_\theta(x)$ sums four priority levels over live deployments only:

$$Q_\theta = \sum_{\pi,t} c^4_{\pi,t} e^{\theta}_{\pi,t} + \sum_{\pi,h,t} c^3_{\pi,h,t} \, \text{cov}^{\theta}_{\pi,h,t} + \sum_{j} c^2_j g^{\theta}_j + \sum_{\phi} c^2_\phi g^{\theta}_\phi + \sum_{i,z,t,p} c^1_{i,p} d^{\theta}_{i,z,t,p}, \tag{3}$$

where cov marks a covered hop, e an interception before the target (L4, weight 1,000), g_j and g_ϕ a detected technique and tactic family (L2, weights 10 and 12), and the last term

Algorithm 1 Certified max–min search

Require: hard clauses $\mathcal{H}$, adder networks with comparators $\ell_\theta(T)$

$lo \leftarrow 0$ ▷ feasible; $hi \leftarrow \min_\theta Q^{\max}_\theta + 1$ ▷ infeasible
while $hi - lo > 1$ **do**
  $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$
  if $\mathcal{H} \wedge \bigwedge_\theta \ell_\theta(mid)$ is SAT with model x **then**
    $lo \leftarrow \max(mid, \min_\theta Q_\theta(x))$; $x^\star \leftarrow x$
  else $hi \leftarrow mid$
  end if
end while
Export $\mathcal{H} \wedge \bigwedge_\theta \ell_\theta(lo + 1)$ and its refutation proof
return $r^\star = lo$, $x^\star$, certificate

basic detection (L1, weight 1); hop coverage (L3) has weight $100 + 70$. Each coefficient is scaled by path frequency, path emphasis, hop value and technique weight, multiplied by 100 and rounded, so every score is exact to 0.01. The robust target is

$$r^\star = \max_{x \text{ feasible}} \min_{\theta \in \Theta} Q_\theta(x). \tag{4}$$

The binding profile depends on the schedule (θ_{high} for 10 of the 13 schedules in Table V, θ_{burst} for 3), so no single profile can be chosen in advance.

Theorem 1. *ZSTP placement is NP-hard.*

Proof. Reduce Maximum k-Coverage [6]: given items U, sets $S_1, \ldots, S_m$ and k, build an instance with one zone, one slot, one role, k fingerprints, slot and zone budget k, one trap type per set, and one technique per item, where type i detects exactly the techniques in S_i. There are no conflicts and no air gap, every type is plausible, every path frequency is 0 (C10a requires nothing), $H = 1 < W$ (C10b has no window), and no churn or burn rule applies. With unit technique weights and all other layer weights 0, a schedule of score v selects at most k types whose sets cover v items, and conversely; each chosen type gets its own fingerprint, so C12 and C14 hold. □

IV. Certified Solution Procedures

A. Certified Max–Min Search (G4)

Algorithm 1 computes $r^\star$. For each profile an adder network [17] computes $Q_\theta(x)$ in binary, and a comparator literal $\ell_\theta(T)$ is true exactly when $Q_\theta(x) \geq T$. An incremental SAT solver (Glucose via PySAT [18]) binary-searches the largest T reached by all four profiles at once. On the instance this takes 17 SAT calls and about 21 s. The certificate is the CNF with $\ell_\theta(r^\star + 1)$ for every θ (in units of 0.01; 52,386 variables, 277,714 clauses) together with its 75,708-line refutation, accepted by the independent checker DRAT-trim [19]. Among the schedules attaining $r^\star$, the one with the highest Q_{med} is reported.

A medium-profile schedule is also computed by four RC2 [20], [21] passes, one per priority level, with each earlier optimum kept as a hard floor: the fixed weights alone do not enforce tier priority (one unit of L4 is worth 433.50 while the lower levels together can add 5,985.68). It serves as a

Algorithm 2 Certified threat-adaptive re-scheduling

Require: deployed schedule x_{dep}; bundle B (campaign, ρ_π, confidence c)

1: $q_p \leftarrow \mathrm{normalize}(q_p + \alpha c[p = p(B)])$, $\alpha = 0.3$
2: update ρ_π, thresholds τ_d, τ_{dp} and slot validity; rebuild encoding
3: $(r^{\star\prime}, x') \leftarrow$ Algorithm 1 with a new certificate
4: **if** x_{dep} follows every rule and $\min_\theta Q'_\theta(x_{\mathrm{dep}}) \geq r^{\star\prime}$ **then** keep x_{dep}
5: **else** redeploy x'
6: **end if**
7: audit every hard rule with the independent checker and verify the certificate before deployment

comparison, not as the deployed schedule; a burst-first variant (maximize Q_{burst}, then the others with floors) is reported likewise.

B. Certified Threat-Adaptive Loop

Algorithm 2 responds to STIX 2.1 [22] intelligence. Alert-driven re-optimization itself follows [1], [3]; the addition is that every re-solve is certified and audited before deployment. On a high-confidence OT bundle (path frequency 0.15 to 0.72, confidence 0.92), the prior of the only OT-plausible role rises from 0.25 to 0.412, τ_d falls from 2 to 1 in DMZ, Mgmt and OT for θ_{low} and θ_{med}, and C10a requires 3 covered slots; the deployed schedule reaches 8,501.83 below the new optimum 9,352.53, so the loop redeploys. The earlier rule of redeploying when Q_{med} rises would have raised Q_{med} by 6.4% while lowering the worst case by 17.2%; the certified re-solve's worst case is 31.4% higher. On a financial bundle the deployed schedule already attains the new optimum (7,052.66) and is kept.

V. Worked Example

A small instance keeps one corridor of the network: the Mgmt zone, where administrators' jump hosts bridge IT and the plant, and the OT zone that controls physical equipment, over $H = 3$ shifts of 8 h. SSH and RDP traps are plausible only in Mgmt; a Modbus/TCP controller is plausible only in OT and only as Generic_Linux. There are two personas (DevOps_JumpHost with fpA, Generic_Linux with fpB), budgets $B_t = 2$, $B_z = 1$, one path Mgmt→OT whose OT hop is critical ($\rho_\pi = 0.5$), and three profiles with $(\tau_d, \tau_{dp}) = (2,3), (2,2), (1,1)$. Of 36 decisions, plausibility fixes 24; the hard rules admit 87 of the $2^{12} = 4{,}096$ assignments, so every number below was also checked by enumeration (Fig. 3).

The static schedule, with the same decoys under the same identities in every shift, loses four of six deployments to recognition under θ_{burst} and scores 1,402 (2,744 under the slower profiles). The max–min schedule changes the Mgmt decoy's type between shifts 1 and 3 and rests fpA in shift 2, so nothing in Mgmt repeats; in OT, C10a and C10b with $\rho_\pi = 0.5$ admit only coverage patterns with adjacent shifts, so exactly one OT deployment is recognized. Its worst case is $r^\star = 2{,}583$, $1.84\times$ the static schedule's. The schedule that is best for θ_{med} scores only 2,382 under θ_{burst}, because it shows fpB in Mgmt and then in OT in consecutive shifts. Adding a third fingerprint to the jump-host role raises the certified optimum to 3,553 (+37.6%): the Mgmt identity rotates fpB→fpC→fpA and the Mgmt decoy stays live in every shift. The example thus shows all four gaps at small scale.

VI. Evaluation

A. Setup

All results come from one pipeline (Python 3.13, PySAT 1.9, OR-Tools CP-SAT 9.15 [23]; two CPU cores; code version 32d54d7e7d96, instance hash 5cb2c5f369c5) and are regenerated by one command. Every schedule is scored by the same independent rule checker and evaluator.

Baselines. Ten heuristics, each respecting the slot budget and the plausibility table but not the other rules unless its own logic happens to: *Greedy-Diverse* (greedy, most ATT&CK techniques first), *Random* (random plausible deployments), *Greedy-BiDir* (greedy additions, then removal of deployments that do not lower the score), *Static-Best* (best one-slot placement repeated), *LP-Relaxation* (linear relaxation of a burn-blind surrogate with budget and per-slot fingerprint limits, rounded per slot), *Max-PathCov* (greedy, hop coverage first), *Round-Robin* (cycles through types and plausible zones), *Single-Zone* (greedy within one zone), *ThreatIntel-Only* (ranks role–type pairs by prior times technique weight, ignoring paths) and *Greedy-HighRho* (covers the hops of the most frequent paths first). Where a heuristic optimizes a score, it uses the burn-blind medium-profile score, as single-scenario heuristics normally do. Each baseline is reported as produced and after minimum-change repair to the nearest rule-compliant schedule (RC2 minimizing the number of changed deployments).

Emulation. Every deployment runs as a TCP server on loopback. In each slot an attacker agent probes each zone with probability ρ_θ, observing every honeypot there, traverses each path with probability $\min(1, \rho_\pi \omega_{\theta,\pi})$, and contacts at each hop every unrecognized honeypot that detects the hop's technique. It recognizes decoys only from banners it actually received, with the thresholds of Section III-D; credit is computed from server-side logs. The random stream depends only on the instance, profile and episode, so every schedule faces the same 100 episodes per profile (common random numbers). The emulator gives a broken rule no attacker-visible effect, which favours rule-breaking schedules.

Per-contribution statistics. For each gap we define a statistic that isolates it, measured in the model and in emulation. Emulation statistics are obtained by replaying all 9,200 recorded episodes from the same seeds with a socket-free simulator of the same attacker that additionally records *why* a decoy was recognized (type repeated in its zone, or fingerprint shown in the previous slots) and *whether* each attack traversal was intercepted at the critical hop; it reproduces the socket emulator's score in every episode (0 mismatches). Differences are oriented so that a positive value favours ZSTP and tested by paired bootstrap (episodes resampled with the same indices for both schedules, independently per profile; 10,000 replicates).

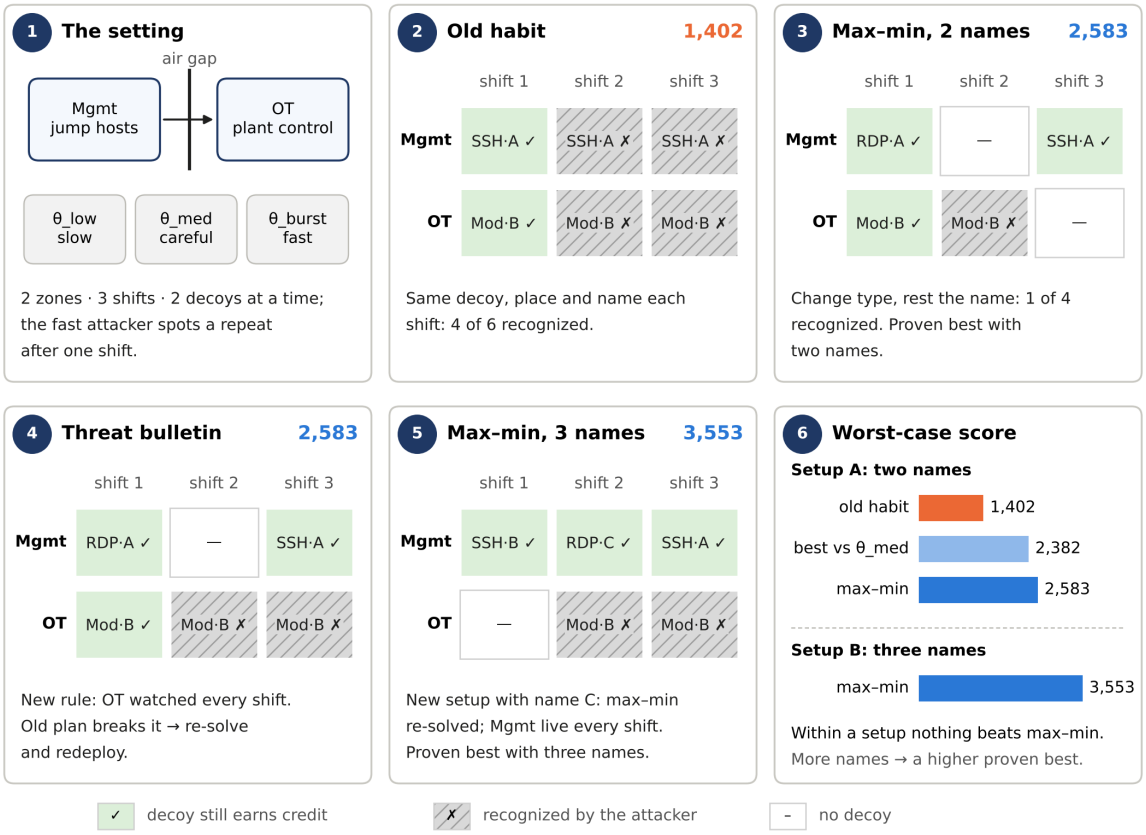


Fig. 3. The worked example. Each timetable shows two zones over three 8-hour shifts; green cells earn credit, hatched cells are recognized by θ_{burst}. The static schedule, the max–min schedule, a threat update that forces a redeployment, and the certified schedule with a third fingerprint; the number at the top right of each step is the worst case over the profiles.

B. Overall Results

Table V and Fig. 4 report every schedule. The max–min schedule attains $r^\star = 7{,}052.62$, confirmed by an independent CP-SAT model, and scores higher than every baseline under all four profiles in the model. All ten baselines break at least one hard rule as produced. Fig. 5 summarizes the four per-contribution tests against each baseline; Sections VI-C–VI-F give the details.

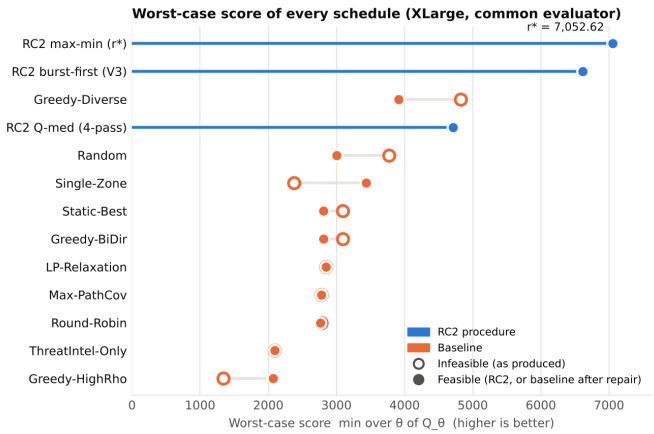


Fig. 4. Worst case of every schedule. Hollow: baseline as produced (breaks a hard rule); filled orange: after minimum-change repair; blue: RC2 procedures. Every repaired baseline stays below $r^\star$.

C. G1: Recognition Against Each Baseline

Table VI measures recognition directly. In the model, θ_{burst} recognizes 27.3% of ZSTP's deployments against 50.0–75.0% of the baselines', and recognition removes 14.4% of the θ_{burst} credit ZSTP's schedule would earn without it, against 32.3–72.6% for the baselines. In emulation the θ_{burst} attacker recognizes 18.8% of ZSTP's deployments against 38.8–57.8%; the paired difference is 20.0–39.0 percentage points with every 95% interval above zero, and stays 24.0–36.6 points after the baselines are repaired. The medium-profile RC2 schedule, which ignores θ_{burst}, is also recognized significantly more often (+13.4 [+11.5, +15.2]); the burst-first schedule, which targets θ_{burst}, matches ZSTP on this statistic but not on the worst case (Section VI-F). Ablation A1 (Section VI-H) confirms the cause: a recognition-blind planner deploys 16 decoys instead of 11, and 75.0% (model) and 53.7% (emulation) of them are recognized.

D. G2: Identity Against Each Baseline

Table VII isolates identity. 7 of the ten baselines show a fingerprint twice in one zone or in two zones at once (C12, C14);

TABLE V
EVERY SCHEDULE UNDER THE COMMON EVALUATOR: WORST CASE WITH BINDING PROFILE, RULES BROKEN AS PRODUCED, AND THE SCORE UNDER EACH PROFILE.

Schedule	Worst case (binding)	Rules broken as produced	Q_{low}	Q_{med}	Q_{high}	Q_{burst}
ZSTP max–min ($r^\star$)	**7,052.62 (high)**	**none**	**7,247**	**9,514**	**7,053**	**7,950**
RC2 medium profile	4,715.04 (high)	none	7,675	10,498	4,715	6,227
RC2 burst-first	6,610.37 (high)	none	6,738	9,606	6,610	8,379
Greedy-Diverse	4,822.52 (high)	C12×2, C14×3	6,093	7,951	4,823	5,389
Random	3,770.69 (burst)	C10a×1, C10b×6, C12×1, C14×4, C3×2, C4×1, C8×1	5,290	4,910	3,889	3,771
Greedy-BiDir	3,093.19 (high)	C14×4	5,326	6,366	3,093	3,344
Static-Best	3,093.19 (high)	C14×4	4,757	6,331	3,093	3,344
LP-Relaxation	2,849.21 (high)	C10a×1, C10b×3	3,997	5,570	2,849	3,186
Max-PathCov	2,782.72 (high)	C10a×1, C10b×3	3,792	5,456	2,783	3,168
Round-Robin	2,776.85 (high)	C10b×2, C12×1, C14×3, C8×3	4,507	5,357	2,777	2,901
Single-Zone	2,376.09 (burst)	C10a×2, C10b×6, C12×2, C3×4	3,487	4,073	3,068	2,376
ThreatIntel-Only	2,090.52 (high)	C10a×3, C10b×9	2,994	4,197	2,091	2,332
Greedy-HighRho	1,341.25 (burst)	C10a×2, C10b×6, C14×4	3,538	3,538	1,654	1,341

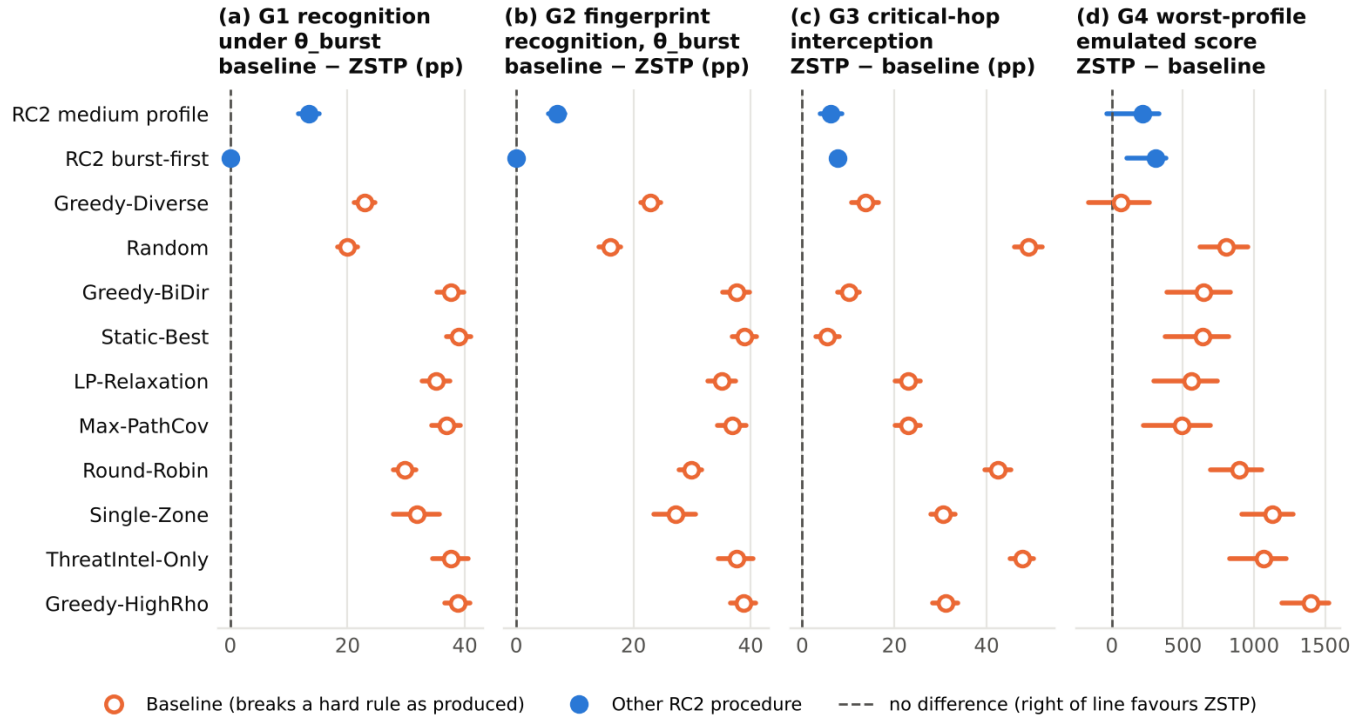


Fig. 5. Each contribution tested against each baseline in emulation (as produced; 95% paired-bootstrap intervals; right of the dashed line favours ZSTP). (a) G1: share of deployments the θ_{burst} attacker recognizes. (b) G2: share recognized with fingerprint reuse as a cause. (c) G3: share of attack traversals, all profiles pooled, intercepted at the critical hop. (d) G4: worst-profile emulated mean.

ZSTP never does. Under θ_{burst} ($\tau_{dp} = 1$) fingerprint reuse is a cause in nearly every recognition: in emulation 18.8% of ZSTP's deployments are recognized with fingerprint reuse as a cause, against 34.9–57.8% for the baselines (difference 16.1–39.0 points, every interval above zero; 23.2–35.9 points after repair). Fingerprint reuse is therefore involved in nearly every recognition by a fast attacker, so managing identity, not only rotating types, decides whether a decoy survives; the persona axis is what makes identity a decision. Giving each role two fingerprints raises $r^\star$ from 7,052.62 to 9,082.56 (+28.8%) and lowers fingerprint burn under θ_{burst} from 27.3% to 18.8% with every rule still satisfied; ablation A2, which removes identity from the model, breaks C14 4 times and has 68.8% of its deployments recognized under θ_{burst}.

E. G3: Temporal Planning Against Each Baseline

Table VIII tests the inter-slot rules. 7 of the ten baselines break C8, C10a or C10b as produced; those that break C10b leave a critical hop uncovered for 3–4 consecutive slots, against at most 1 for ZSTP. In emulation, ZSTP intercepts the attacker at the critical hop in 73.8% of traversals, against 24.7–68.3% for the baselines; the difference is 5.5–49.1 points with every interval above zero, and 7.1–37.1 points after repair.

TABLE VI

G1 AGAINST EACH BASELINE. MODEL: SHARE OF DEPLOYMENTS BURNED UNDER θ_{burst}; SHARE OF ATTAINABLE θ_{burst} CREDIT LOST TO RECOGNITION; AVERAGE FORCE MULTIPLIER UNDER θ_{burst}. EMULATION: SHARE OF DEPLOYMENTS THE θ_{burst} ATTACKER RECOGNIZES; DIFFERENCE = BASELINE MINUS ZSTP (PERCENTAGE POINTS, 95% PAIRED BOOTSTRAP).

Schedule	Burned (%)	Credit lost (%)	FM-avg	Emul. recognized (%)	Baseline − ZSTP [95% CI]
ZSTP max–min ($r^\star$)	**27.3**	**14.4**	**1.11**	**18.8**	**—**
RC2 medium profile	45.5	41.8	0.84	32.2	+13.4 [+11.5, +15.2]
RC2 burst-first	27.3	18.2	1.11	18.8	+0.0 [+0.0, +0.0]
Greedy-Diverse	56.2	53.5	0.59	41.8	+22.9 [+21.2, +24.6]
Random	50.0	32.3	0.37	38.8	+20.0 [+18.2, +21.7]
Greedy-BiDir	75.0	72.6	0.41	56.4	+37.6 [+35.3, +39.8]
Static-Best	75.0	72.5	0.41	57.8	+39.0 [+36.9, +41.0]
LP-Relaxation	75.0	71.6	0.41	53.9	+35.1 [+32.7, +37.5]
Max-PathCov	75.0	72.1	0.45	55.6	+36.8 [+34.3, +39.2]
Round-Robin	62.5	59.1	0.29	48.6	+29.8 [+27.8, +31.7]
Single-Zone	68.8	60.2	0.36	50.6	+31.8 [+27.8, +35.7]
ThreatIntel-Only	75.0	72.6	0.34	56.4	+37.6 [+34.5, +40.5]
Greedy-HighRho	75.0	67.9	0.25	57.6	+38.7 [+36.5, +40.9]

TABLE VII

G2 AGAINST EACH BASELINE. MODEL: IDENTITY-RULE VIOLATIONS AS PRODUCED, CROSS-ZONE FINGERPRINT LEAKS, SHARE OF DEPLOYMENTS WITH FINGERPRINT BURN UNDER θ_{burst}. EMULATION: SHARE OF DEPLOYMENTS RECOGNIZED WITH FINGERPRINT REUSE AS A CAUSE UNDER θ_{burst}; DIFFERENCE = BASELINE MINUS ZSTP (PERCENTAGE POINTS, 95% PAIRED BOOTSTRAP).

Schedule	C12	C14	Leaks	FP burn (%)	Emul. FP-recognized (%)	Baseline − ZSTP [95% CI]
ZSTP max–min ($r^\star$)	**0**	**0**	**0**	**27.3**	**18.8**	**—**
RC2 medium profile	0	0	0	36.4	25.8	+7.0 [+5.5, +8.4]
RC2 burst-first	0	0	0	27.3	18.8	+0.0 [+0.0, +0.0]
Greedy-Diverse	2	3	3	56.2	41.8	+22.9 [+21.2, +24.6]
Random	1	4	6	43.8	34.9	+16.1 [+14.2, +17.8]
Greedy-BiDir	0	4	4	75.0	56.4	+37.6 [+35.3, +39.8]
Static-Best	0	4	4	75.0	57.8	+39.0 [+36.9, +41.0]
LP-Relaxation	0	0	0	75.0	53.9	+35.1 [+32.7, +37.5]
Max-PathCov	0	0	0	75.0	55.6	+36.8 [+34.2, +39.2]
Round-Robin	1	3	3	62.5	48.6	+29.8 [+27.9, +31.7]
Single-Zone	2	0	0	62.5	45.9	+27.1 [+23.5, +30.6]
ThreatIntel-Only	0	0	0	75.0	56.4	+37.6 [+34.4, +40.5]
Greedy-HighRho	0	4	4	75.0	57.6	+38.7 [+36.6, +40.9]

The rule-breaking baselines reach only 24.7–50.9%, the rule-following ones 60.2–68.3%. Both RC2 variants follow the rules but still intercept significantly less often (medium profile +6.2 [+3.9, +8.6], burst-first +7.6 [+6.4, +8.9]): following the rules is necessary but not sufficient, and the joint worst-case plan does better. The rules also interact with recognition: zero recognition under all four profiles is infeasible (exact CP-SAT check), because the only OT decoy that covers the critical hop must stand in adjacent slots and the churn limit rules out alternating it; with the churn limit relaxed to 3, zero recognition becomes feasible but costs 15.8% of the worst case (6,196.16 against 7,360.66). Ablation A3, which re-solves one slot at a time without these rules, leaves a critical hop uncovered too long (C10b×3; path persistence 25%).

F. G4: Assurance Against Each Baseline

Table XI compares worst cases. No baseline reaches $r^\star$, as produced or repaired: $r^\star$ is $1.46\times$ the best baseline's worst case as produced and $1.80\times$ after repair. In emulation the max–min schedule has the highest worst-profile mean (2,030) and is significantly higher than 20 of the 22 other schedules, including every repaired baseline; the differences from Greedy-Diverse as produced, which breaks the identity rules, and from the medium-profile schedule are not significant. Heuristics that target the worst case directly come closer but cannot certify it (Table IX): a robust local search reaches $r^\star$ in 1 of 11 restarts, and nothing in its output distinguishes that run from the others. The contribution is therefore the certified optimum, not a score no heuristic could find.

Profile by profile the emulated picture is mixed, as expected of a max–min objective, which protects the weakest profile rather than each profile (Table X). Under θ_{high} 1 and under θ_{burst} 1 of the 22 other schedules have a higher emulated mean than the max–min schedule, but under θ_{low} 11 and under θ_{med} 8 do. Under the Bernoulli attacker the max–min schedule's weakest profile is θ_{low} rather than θ_{high}, because a low-rate attacker probes few honeypots per episode; the emulation complements the model rather than reproducing its worst case.

TABLE VIII

G3 AGAINST EACH BASELINE. MODEL: INTER-SLOT RULE VIOLATIONS AS PRODUCED; LONGEST RUN OF SLOTS IN WHICH A CRITICAL HOP IS UNCOVERED; SHARE OF PATHS MEETING C10A AND C10B; EARLY INTERCEPTION AT θ_{med}. EMULATION: SHARE OF ATTACK TRAVERSALS (ALL PROFILES POOLED) INTERCEPTED AT THE CRITICAL HOP; DIFFERENCE = ZSTP MINUS BASELINE (PERCENTAGE POINTS, 95% PAIRED BOOTSTRAP).

Schedule	C8	C10a	C10b	Longest gap	Persist. (%)	Early int. (%)	Emul. critical-hop (%)	ZSTP − baseline [95% CI]
ZSTP max–min ($r^\star$)	**0**	**0**	**0**	**1**	**100**	**81.2**	**73.8**	**—**
RC2 medium profile	0	0	0	1	100	93.8	67.6	+6.2 [+3.9, +8.6]
RC2 burst-first	0	0	0	1	100	87.5	66.2	+7.6 [+6.4, +8.9]
Greedy-Diverse	0	0	0	1	100	68.8	60.2	+13.6 [+10.7, +16.6]
Random	1	1	6	4	25	43.8	24.7	+49.1 [+46.1, +52.2]
Greedy-BiDir	0	0	0	1	100	50.0	63.7	+10.1 [+7.7, +12.4]
Static-Best	0	0	0	0	100	50.0	68.3	+5.5 [+3.0, +8.0]
LP-Relaxation	0	1	3	4	75	37.5	50.9	+22.9 [+20.2, +25.6]
Max-PathCov	0	1	3	4	75	37.5	50.9	+22.9 [+20.2, +25.6]
Round-Robin	3	0	2	3	75	43.8	31.4	+42.4 [+39.7, +45.2]
Single-Zone	0	2	6	4	50	25.0	43.3	+30.5 [+27.9, +33.1]
ThreatIntel-Only	0	3	9	4	25	37.5	26.1	+47.7 [+45.2, +50.2]
Greedy-HighRho	0	2	6	4	50	25.0	42.7	+31.1 [+28.4, +33.8]

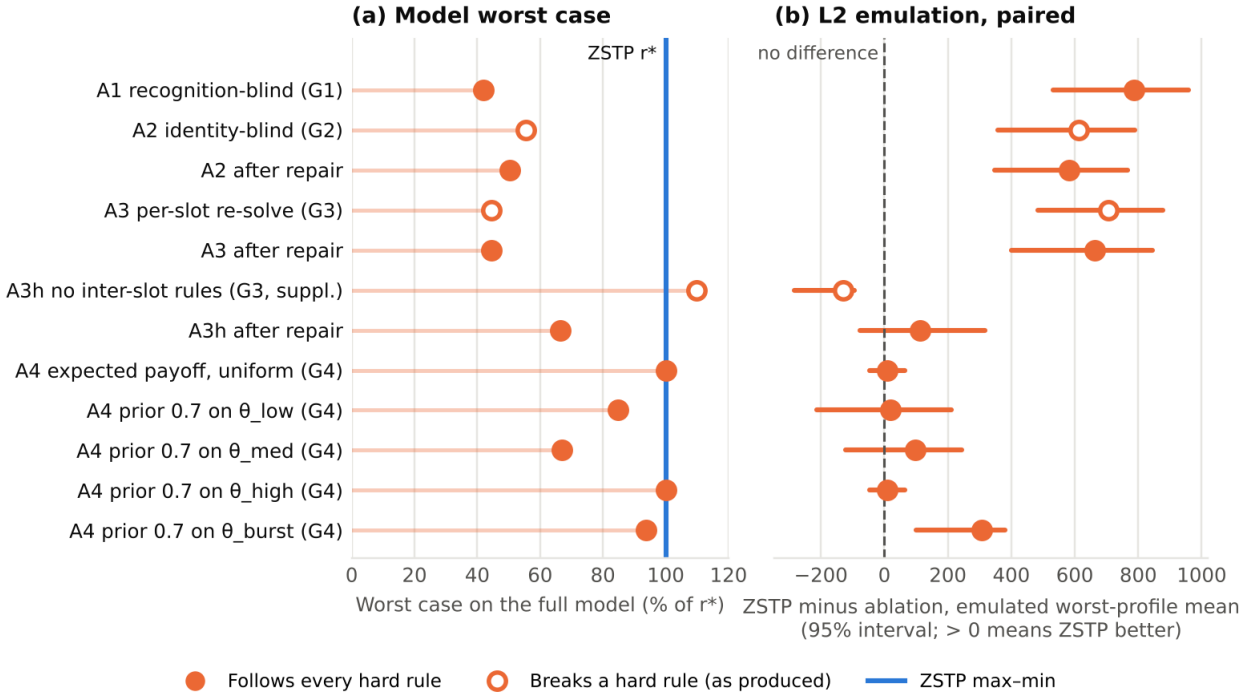


Fig. 6. Per-gap ablations. (a) Worst case on the full model as a share of $r^\star$. (b) Emulated worst-profile mean, max–min minus ablation, with 95% paired-bootstrap intervals. Hollow markers break a hard rule as produced.

G. Fairness: Rule-Compliant Baselines

The baselines as produced break hard rules, and the emulator does not penalize a broken rule, so a comparison could favour either side. Table XIII therefore repeats all four per-contribution tests against each baseline after minimum-change repair to the nearest rule-compliant schedule. ZSTP keeps a significant advantage on every statistic against every repaired baseline: recognition under θ_{burst} is lower by 24.0–36.6 points (G1), recognition through fingerprint reuse by 23.2–35.9 points (G2), critical-hop interception is higher by 7.1–37.1 points (G3), and the worst-profile emulated mean is higher by 352–942 points (G4), with every 95% interval above zero. Repair raises critical-hop interception for all seven baselines that broke an inter-slot rule, as expected, since it restores those rules; it does not close any gap, because repair restores rules without modeling recognition or identity.

H. Per-Gap Ablations

The baseline comparisons show that ZSTP is better on each gap's statistic; the ablations show that each capability is the cause. Each ablation removes the single capability that closes one gap and re-solves with the procedure used for $r^\star$: A1 plans without recognition; A2 without identity (no C12, C14, no fingerprint burn); A3 re-solves one slot at a time without

TABLE IX
TOP: WORST-CASE-OPTIMIZING HEURISTICS AGAINST THE CERTIFIED OPTIMUM. BOTTOM: SCALING OF THE CERTIFIED SEARCH (EVERY PROOF VERIFIED).

Method	Worst case	% of $r^\star$	Proven
RC2 max–min (certified)	7,052.62	100.0	yes
Robust local search, best of 11	7,052.62	100.0	no
Robust local search, mean of 11	6,039.21	85.6	no
Robust greedy (repaired)	5,821.95	82.6	no
Best burn-blind baseline (repaired)	3,915.24	55.5	no
Instance	**Decisions**	$r^\star$	**Time (s)**
$H = 4$, $\lvert F\rvert = 4$	640	7,052.62	20.8
$H = 5$, $\lvert F\rvert = 4$	800	7,738.61	75.5
$H = 6$, $\lvert F\rvert = 4$	960	9,071.80	55.4
$H = 8$, $\lvert F\rvert = 4$	1,280	9,809.98	919.9
$H = 4$, $\lvert F\rvert = 8$ (2 per role)	1,280	9,082.56	122.8

TABLE X
EMULATED MEAN SCORE UNDER EACH PROFILE (100 EPISODES, COMMON RANDOM NUMBERS; * LOWEST PROFILE OF EACH SCHEDULE), AS PRODUCED.

Schedule	Low	Med	High	Burst
ZSTP max–min	**2,030***	**2,614**	**2,075**	**2,282**
RC2 medium profile	2,088	2,773	1,817*	2,131
RC2 burst-first	1,724*	2,484	2,054	2,588
Greedy-Diverse	2,418	2,976	2,097	1,966*
Random	1,547	1,777	1,236	1,229*
Greedy-BiDir	2,493	2,888	1,664	1,384*
Static-Best	2,471	2,886	1,713	1,393*
LP-Relaxation	2,136	2,658	1,776	1,469*
Max-PathCov	1,987	2,589	1,949	1,538*
Round-Robin	1,286	1,771	1,212	1,136*
Single-Zone	1,936	2,132	1,634	900*
ThreatIntel-Only	1,609	2,027	1,323	961*
Greedy-HighRho	2,185	2,214	1,024	631*

C8, C10a and C10b, as the 2026 papers re-solve per round or alert; A4 replaces the worst case by the expected payoff $\sum_\theta \pi_\theta Q_\theta$, solved exactly by the same SAT search under a uniform prior and under four priors that put 0.7 on one profile (the uniform optimum is confirmed by CP-SAT). With nothing removed, the ablation encoder produces exactly the 33,011 hard clauses of the main encoding. Every resulting schedule is checked, scored, repaired if needed and emulated like the baselines; two complete runs gave identical results (Table XII, Fig. 6).

Leaving recognition, identity or multi-slot planning out cuts the worst case to 42.0–55.5% of $r^\star$ and the emulated worst-profile mean by 583–787 points, every interval above zero, as produced and after repair. Without the inter-slot rules but with the full horizon (A3h), the schedule scores above $r^\star$ (109.9%) and above ZSTP in emulation (-130 [-284, -94]) because it breaks churn and gap rules that operators impose; repaired, it falls to 66.4% of $r^\star$ and its emulated difference (+114 [-77, +317]) is not significant. The rules cost score, and ZSTP is the best schedule that respects them. For G4, the expected-payoff optimum reaches $r^\star$ under the uniform prior and under the prior weighted toward θ_{high}, the binding profile, with an emulated difference that is not significant (+8 [-47, +64]); under priors weighted toward θ_{low}, θ_{med} and θ_{burst} it loses 15.2%, 33.1% and 6.3% of the worst case, significantly in emulation only for θ_{burst} (+306 [+101, +379]). An expected-payoff schedule matches the worst case only when its prior happens to favour the binding profile, which cannot be known without computing $r^\star$.

I. Scaling and Sensitivity

The certified search stays tractable as the horizon grows (Table IX): up to 76 s for $H \leq 6$ and 15 minutes at $H = 8$ (1,280 decisions) on two cores, with every proof verified; solve time grows steeply with the horizon and is not monotone, because the number and difficulty of SAT calls vary. Under $\pm 25\%$ changes to path frequencies, hop values, technique weights and the L4 weight, the original max–min schedule stays within 0.19% of the re-solved optimum in 7 of 8 cases and within 10.18% when all path frequencies rise by 25%; after a shift of the Finance role prior to 0.40 it remains optimal.

J. Validation of the Encoding and Emulation

A certificate is only as sound as the encoding it certifies. Table XIV lists the independent tests; none found a disagreement.

K. Threats to Validity

Internal. The encoding is tested against an independently written checker and evaluator in both directions, an independent CP-SAT model and exhaustive enumeration (Table XIV). *Construct.* The technique map, attack-path steps, plausibility tables, path emphasis and layer weights are documented defaults, not measured values; Section VI-I reports the sensitivity. *External.* Results come from one instance with four discrete profiles; the scaling study varies the horizon and fingerprints but not the network, and the attacker is a Bernoulli process on loopback sockets, not real attacker tooling. *Baselines.* The heuristics optimize a burn-blind, single-profile score and are not given every hard rule, which favours ZSTP; every comparison is therefore also reported after repair. The 2026 methods [1], [3] are not re-implemented; their objectives are represented by ablations A3 and A4 on the same model. *Statistical.* Two of the 22 worst-profile emulation comparisons and the G4 ablation comparisons noted above are not significant and are reported as such.

VII. DISCUSSION

What the gaps mean for deployment. The per-contribution tests point to concrete operating rules. Recognition, not coverage, is what separates the schedules: every baseline covers paths, but loses most of its decoys to a fast attacker (G1). Fingerprint reuse is involved in nearly every recognition by such an attacker, so the size of the fingerprint catalogue is a first-order design parameter: one extra fingerprint per role is worth 28.8% of the worst case on our instance (G2). Coverage rules on critical hops should be treated as policy rather than as score terms, because a planner without them gains score only by breaking them (A3h), and with them the schedule must repeat exactly where recognition is fastest; ZSTP computes the

TABLE XI

G4 AGAINST EACH BASELINE. WORST CASE AS PRODUCED AND AFTER MINIMUM-CHANGE REPAIR (CHANGED DEPLOYMENTS); $r^\star$ DIVIDED BY THE REPAIRED WORST CASE; EMULATED WORST-PROFILE MEAN, MAX–MIN MINUS SCHEDULE (95% PAIRED BOOTSTRAP), AS PRODUCED AND REPAIRED.

Schedule	Worst case	Repaired (changes)	$r^\star$/repaired	Emul. diff., as produced	Emul. diff., repaired
ZSTP max–min ($r^\star$)	**7,052.62**	**feasible**	**1.00**	**—**	**—**
RC2 medium profile	4,715.04	feasible	1.50	+214 [−33, +333]	—
RC2 burst-first	6,610.37	feasible	1.07	+306 [+104, +379]	—
Greedy-Diverse	4,822.52	3,915.24 (5)	1.80	+64 [−166, +265]	+352 [+163, +553]
Random	3,770.69	3,002.83 (13)	2.35	+801 [+622, +958]	+942 [+706, +1,092]
Greedy-BiDir	3,093.19	2,806.43 (4)	2.51	+646 [+387, +836]	+700 [+463, +885]
Static-Best	3,093.19	2,806.43 (4)	2.51	+638 [+372, +822]	+672 [+408, +857]
LP-Relaxation	2,849.21	2,849.21 (4)	2.48	+561 [+298, +745]	+590 [+331, +772]
Max-PathCov	2,782.72	2,782.72 (4)	2.53	+492 [+220, +694]	+523 [+249, +722]
Round-Robin	2,776.85	2,764.98 (10)	2.55	+894 [+693, +1,053]	+763 [+569, +941]
Single-Zone	2,376.09	3,432.89 (12)	2.05	+1,130 [+912, +1,274]	+681 [+422, +862]
ThreatIntel-Only	2,090.52	2,090.52 (12)	3.37	+1,070 [+831, +1,229]	+884 [+625, +1,056]
Greedy-HighRho	1,341.25	2,070.51 (10)	3.41	+1,399 [+1,195, +1,525]	+779 [+537, +958]

TABLE XII

PER-GAP ABLATIONS ON THE FULL MODEL. BURST BURN: SHARE OF DEPLOYMENTS RECOGNIZED UNDER θ_{burst}. EMULATION: WORST-PROFILE MEAN OVER 100 EPISODES PER PROFILE; DIFFERENCE: MAX–MIN MINUS ABLATION, 95% PAIRED BOOTSTRAP. †SUPPLEMENTARY. THE θ_{burst}-WEIGHTED A4 OPTIMUM HAS THE SAME MODEL SCORES AS THE BURST-FIRST SCHEDULE; ITS INTERVAL DIFFERS SLIGHTLY FROM TABLE XI ([+101, +379] VS. [+104, +379]) BECAUSE THE TWO BOOTSTRAP RUNS DRAW INDEPENDENT RESAMPLES.

Gap	Ablation	Rules broken	Worst case	% of $r^\star$	Burst burn %	Emul. mean	Max–min minus ablation
—	**ZSTP max–min (full model)**	**none**	**7,052.62**	**100.0**	**27.3**	**2,030**	**—**
G1	A1 recognition-blind planner	none	2,964.92	42.0	75.0	1,243	+787 [+532, +957]
G2	A2 identity-blind planner	C14×4	3,915.09	55.5	68.8	1,416	+614 [+357, +790]
G2	after repair (7 changes)	none	3,546.53	50.3	63.6	1,447	+583 [+347, +766]
G3	A3 per-slot re-solve, no inter-slot rules	C10b×3	3,133.91	44.4	66.7	1,325	+705 [+483, +876]
G3	after repair (4 changes)	none	3,133.91	44.4	71.4	1,367	+663 [+400, +844]
G3	A3h full horizon, no inter-slot rules†	C10b×2, C8×3	7,752.55	109.9	27.3	2,161	−130 [−284, −94]
G3	after repair (6 changes)	none	4,681.42	66.4	61.5	1,916	+114 [−77, +317]
G4	A4 expected payoff, uniform prior	none	7,052.62	100.0	27.3	2,022	+8 [−47, +64]
G4	prior 0.7 on θ_{low}	none	5,982.84	84.8	53.8	2,012	+18 [−214, +210]
G4	prior 0.7 on θ_{med}	none	4,715.04	66.9	50.0	1,934	+96 [−122, +243]
G4	prior 0.7 on θ_{high}	none	7,052.62	100.0	27.3	2,022	+8 [−47, +64]
G4	prior 0.7 on θ_{burst}	none	6,610.37	93.7	27.3	1,724	+306 [+101, +379]

TABLE XIII

ALL FOUR PER-CONTRIBUTION TESTS AGAINST EACH BASELINE AFTER MINIMUM-CHANGE REPAIR (EVERY SCHEDULE FOLLOWS EVERY HARD RULE). DIFFERENCES FAVOUR ZSTP WHEN POSITIVE; 95% PAIRED BOOTSTRAP. ZSTP: 18.8% RECOGNIZED, 18.8% FINGERPRINT-RECOGNIZED, 73.8% CRITICAL-HOP INTERCEPTION, WORST-PROFILE MEAN 2,030.

Repaired baseline (changes)	G1 recog. (%)	G1: B − ZSTP	G2 FP recog. (%)	G2: B − ZSTP	G3 crit. (%)	G3: ZSTP − B	G4: ZSTP − B
Greedy-Diverse (5)	46.3	+27.5 [+25.0, +29.7]	46.3	+27.5 [+25.1, +29.7]	55.9	+17.9 [+14.8, +21.1]	+352 [+163, +553]
Random (13)	44.9	+26.1 [+23.5, +28.6]	44.9	+26.1 [+23.5, +28.5]	43.0	+30.8 [+27.8, +33.9]	+942 [+706, +1,092]
Greedy-BiDir (4)	53.7	+34.8 [+32.6, +37.0]	53.7	+34.8 [+32.6, +37.0]	57.5	+16.3 [+14.2, +18.4]	+700 [+463, +885]
Static-Best (4)	54.2	+35.4 [+32.8, +37.9]	54.2	+35.4 [+32.9, +38.0]	66.7	+7.1 [+4.9, +9.2]	+672 [+408, +857]
LP-Relaxation (4)	52.9	+34.1 [+31.4, +36.7]	52.9	+34.1 [+31.5, +36.8]	57.2	+16.6 [+13.8, +19.3]	+590 [+331, +772]
Max-PathCov (4)	54.6	+35.8 [+33.4, +38.2]	54.6	+35.8 [+33.4, +38.1]	57.2	+16.6 [+13.8, +19.3]	+523 [+249, +722]
Round-Robin (10)	42.8	+24.0 [+22.0, +26.0]	42.0	+23.2 [+21.0, +25.3]	36.7	+37.1 [+33.6, +40.5]	+763 [+569, +941]
Single-Zone (12)	47.9	+29.1 [+26.6, +31.6]	47.9	+29.1 [+26.5, +31.6]	55.0	+18.8 [+15.7, +21.9]	+681 [+422, +862]
ThreatIntel-Only (12)	55.4	+36.6 [+34.2, +38.9]	54.8	+35.9 [+33.6, +38.2]	45.5	+28.3 [+25.2, +31.5]	+884 [+625, +1,056]
Greedy-HighRho (10)	51.4	+32.5 [+30.0, +35.0]	51.4	+32.5 [+30.1, +35.0]	54.0	+19.8 [+16.6, +23.0]	+779 [+537, +958]

best trade-off and proves it (G3). Finally, when the attacker mix is uncertain, an expected-payoff objective is safe only if its prior happens to favour the binding profile; the max–min objective needs no prior (G4).

Relation to attack-graph methods. ZSTP is complementary to the 2026 placement work rather than a replacement: attack-graph ranking [1] and reduction [2] supply the paths, Bayesian belief updates [3] can feed the persona priors of Algorithm 2, and ZSTP adds recognition, identity and temporal structure with a certified worst case.

Limits. The guarantee covers the encoded model and the four modeled profiles; it says nothing about attackers outside this set. Exact certification grows steeply with the horizon (15 minutes at 1,280 decisions), so larger networks will need

TABLE XIV
VALIDATION TESTS (FINAL CODE; EVERY VALUE WRITTEN TO RESULTS FILES BY THE SINGLE REPRODUCE COMMAND).

Test	Scope	Result
Adder pseudo-Boolean encoding	120 random weighted sums (1–8 literals, weights up to 90,000); every assignment × every threshold	0 errors
Encoding vs. rule checker, both directions	1,635 schedules drawn without the solver (random subsets, one-step mutations, raw baselines)	0 feasibility and 0 score mismatches
Independent CP-SAT model	$r^\star$, the four tier optima, weighted optimum, expected-payoff optimum	all OPTIMAL and equal
Exhaustive enumeration, small instance	262,144 subsets, 131 feasible	$r^\star = 2{,}249.09$ by enumeration, SAT and CP-SAT
Optimality proofs	XLarge $r^\star$, small-instance $r^\star$, both Algorithm 2 re-solves	4 of 4 accepted by DRAT-trim [19]
Deterministic emulation replay	24 schedules × 4 profiles, 2,349 connections	log score = model score in 96 of 96
Reference simulator	every stochastic episode replayed from the same seeds	0 score mismatches
Attacker draw frequencies	20 binomial tests against ρ_θ and $\min(1, \rho_\pi \omega_{\theta,\pi})$	max $\lvert z \rvert = 2.24$ (Bonferroni bound 3.02)
NP-hardness reduction	30 random Maximum k-Coverage instances	optimum = brute force in 30 of 30
Ablation encoder	nothing removed vs. main encoder	identical 33,011 hard clauses
Reproducibility	two complete runs of the pipeline and of the ablations	identical apart from timing

decomposition or warm starts, and a proof of the decomposed problem. Validation is at the process level; VM-level validation with segmented zones and real attacker tooling, and instances with more zones and paths, are the next steps. A distributionally robust variant over an ambiguity set of attacker priors would replace the discrete Θ.

VIII. CONCLUSION

Attack-path coverage, exact placement, multiple attacker types and alert-driven re-optimization are established in the 2026 honeypot-placement literature. ZSTP closes the four gaps that remain: it models per-decoy recognition, treats decoy identity as a decision with hard identity rules, plans a multi-slot schedule under inter-slot rules, and certifies the worst case over the modeled attacker profiles with an independently verified proof. Tested contribution by contribution against ten baselines, ZSTP has its decoys recognized significantly less often (G1, G2), intercepts significantly more attacks at the critical hop (G3) and alone attains the certified worst case (G4); per-gap ablations show that each capability is the cause, and that an expected-payoff objective reaches $r^\star$ only under priors that favour the binding profile.

APPENDIX A
EVALUATED INSTANCE

Tables XV–XVII list the evaluated instance as built by the released code (instance hash 5cb2c5f369c5). Values not taken from earlier work, such as the technique map, path steps and path emphasis, are documented defaults (Section VI-I).

DATA AND CODE AVAILABILITY

The implementation, results files, optimality certificates (CNF and proof files), STIX bundles, the per-contribution test script and the ablation script are provided as supplementary material; `python3 reproduce.py` regenerates every number, table and figure.

TABLE XV
ATTACK PATHS: HOPS AS (ZONE, ATT&CK TECHNIQUE, INTERCEPTION VALUE); THE CRITICAL HOP IS MARKED *; ρ_π IS THE PATH FREQUENCY.

Path	Hops	ρ_π
π_1 web_to_db	(DMZ, T1190, 1), (Internal, T1021, 1.8)*, (Internal, T1213, 2)	0.30
π_2 ot_infiltration	(DMZ, T1133, 1), (Mgmt, T1021, 1.4), (OT, T0886, 1.7)*, (OT, T0855, 2)	0.15
π_3 brute_to_ad	(DMZ, T1110, 1), (Internal, T1558, 1.6)*, (Internal, T1003, 1.9)	0.20
π_4 cloud_exfil	(Cloud, T1190, 1), (Cloud, T1005, 1.3)*, (Cloud, T1048, 1.5)	0.25

TABLE XVI
TRAP TYPES, THE TECHNIQUES EACH DETECTS, AND THE ZONES WHERE EACH IS PLAUSIBLE (G_K); OT IS AIR-GAPPED AGAINST IT-CROSSING TYPES.

Type	Techniques	Plausible zones
ssh_trap	T1110, T1021, T1133, T1570	DMZ, Internal, Cloud, Mgmt
db_trap	T1213, T1005, T1003	Internal, Cloud
web_trap	T1190, T1071, T1046	DMZ, Internal, Cloud
dns_trap	T1572, T1048, T1018	DMZ, Cloud, Mgmt
smb_trap	T1021, T1210, T1570, T1087	Internal
rdp_trap	T1133, T1021, T1558	DMZ, Internal, Mgmt
ot_trap	T0855, T0886, T1046	OT
generic_trap	T1046, T1018, T1087	DMZ, Internal, OT, Mgmt

TABLE XVII
ATTACKER PROFILES: STRENGTH ρ_θ, RECOGNITION THRESHOLDS AFTER THE THREAT UPDATE, AND PATH EMPHASIS $\omega_{\theta,\pi}$.

Profile	ρ_θ	τ_d	τ_{dp}	ω_{π_1}	ω_{π_2}	ω_{π_3}	ω_{π_4}
θ_{low}	0.10	2	3	1	0.2	0.6	1
θ_{med}	0.30	2	2	1	1	1	1
θ_{high}	0.60	1	1	0.8	0.6	1.4	0.8
θ_{burst}	0.85	1	1	0.6	2	1	0.6

REFERENCES

[1] R. Liu, G. Xu, and Z. Hu, "Graph neural network-guided TrapManager for critical path identification and decoy deployment," *Mathematics*, vol. 14, no. 4, Art. no. 683, 2026, doi:10.3390/math14040683.

[2] J. Lacmou Zeutouo, H. S. Kenmeugne Djouko, Y. F. Yankam, A. L. Nguemkam, and V. Kengne Tchendji, "Weighted core attack graph for honeypot allocation against lateral movement in advanced persistent threats," in *Proc. 3rd Int. Conf. Artificial Intelligence and Smart Appli-*

cations (AISA'25), Vol. 2, LNNS vol. 1851, Springer, 2026, pp. 49–58, doi:10.1007/978-3-032-18860-1_5.

[3] D. Park and G. G. Dagher, "Adaptive honeypot allocation in multi-attacker networks via Bayesian Stackelberg games," *Computers & Security*, vol. 168, Art. no. 104949, 2026, doi:10.1016/j.cose.2026.104949.

[4] K. Voznyi, A. Kashtalian, B. Savenko, and V. Lysak, "Honeypot placement for joint detection and attacker-type inference: A Bayesian Stackelberg formulation," in *Proc. 16th Int. Conf. Advanced Computer Information Technologies (ACIT)*, 2026, pp. 846–851, doi:10.1109/ACIT69831.2026.11700210.

[5] M. Zhang, Y. Yu, S. Li, Y. Liu, Y. Zhang, R. Zhang, and S. Shao, "Adaptive software-defined honeypot strategy using Stackelberg game and deep reinforcement learning with DPU acceleration," *Modelling*, vol. 7, no. 1, Art. no. 23, 2026, doi:10.3390/modelling7010023.

[6] U. Feige, "A threshold of $\ln n$ for approximating set cover," *J. ACM*, vol. 45, no. 4, pp. 634–652, 1998.

[7] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2003.

[8] N. Provos, "A virtual honeypot framework," in *Proc. 13th USENIX Security Symp.*, 2004.

[9] G. Wagener, R. State, T. Engel, and A. Dulaunoy, "Adaptive and self-configurable honeypots," in *Proc. 12th IFIP/IEEE Int. Symp. Integrated Network Management*, 2011, pp. 345–352, doi:10.1109/INM.2011.5990710.

[10] C. Kiekintveld, V. Lisý, and R. Píbil, "Game-theoretic foundations for the strategic use of honeypots in network security," in *Cyber Warfare*, Advances in Information Security, vol. 56, Springer, 2015, doi:10.1007/978-3-319-14039-1_5.

[11] J. Pawlick, E. Colbert, and Q. Zhu, "A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy," *ACM Comput. Surv.*, vol. 52, no. 4, Art. 82, 2019, doi:10.1145/3337772.

[12] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Decision and Game Theory for Security (GameSec)*, LNCS 8252, Springer, 2013, doi:10.1007/978-3-319-02786-9_15.

[13] L. Huang and Q. Zhu, "Adaptive honeypot engagement through reinforcement learning of semi-Markov decision processes," in *Decision and Game Theory for Security (GameSec)*, LNCS 11836, 2019, pp. 196–216, doi:10.1007/978-3-030-32430-8_13.

[14] K. Hammar and R. Stadler, "Finding effective security strategies through reinforcement learning and self-play," in *Proc. 16th Int. Conf. Network and Service Management (CNSM)*, 2020, doi:10.23919/CNSM50824.2020.9269092.

[15] S. Noel and S. Jajodia, "Optimal IDS sensor placement and alert prioritization using attack graphs," *J. Netw. Syst. Manage.*, vol. 16, no. 3, pp. 259–275, 2008, doi:10.1007/s10922-008-9109-x.

[16] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix," 2024. [Online]. Available: https://attack.mitre.org/

[17] N. Eén and N. Sörensson, "Translating pseudo-Boolean constraints into SAT," *J. Satisfiability, Boolean Modeling and Computation*, vol. 2, pp. 1–26, 2006.

[18] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python toolkit for prototyping with SAT oracles," in *Theory and Applications of Satisfiability Testing (SAT)*, LNCS 10929, 2018, pp. 428–437, doi:10.1007/978-3-319-94144-8_26.

[19] N. Wetzler, M. J. H. Heule, and W. A. Hunt Jr., "DRAT-trim: Efficient checking and trimming using expressive clausal proofs," in *Theory and Applications of Satisfiability Testing (SAT)*, LNCS 8561, 2014, pp. 422–429, doi:10.1007/978-3-319-09284-3_31.

[20] A. Ignatiev, A. Morgado, and J. Marques-Silva, "RC2: An efficient MaxSAT solver," *J. Satisfiability, Boolean Modeling and Computation*, vol. 11, no. 1, pp. 53–64, 2019, doi:10.3233/SAT190116.

[21] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. 28th AAAI Conf. Artificial Intelligence*, 2014.

[22] OASIS, "STIX Version 2.1," OASIS Standard, 2021.

[23] L. Perron and F. Didier, "CP-SAT," Google OR-Tools, version 9.15. [Online]. Available: https://developers.google.com/optimization/